

多媒體技術與應用

期中考試

midterm

班級：資工三

姓名：楊子力

學號：112370102

日期：2026.04.22 ~ 2026.04.24

書面報告撰寫部分

1. 使用之模型參數與設計邏輯

因為本次期中規定不能使用資料增強，CIFAR-100 非常容易發生 Overfitting。為此，我將重點放在加深網路與壓制過擬合：

- I. 架構加深與正規化：我將 ResNet 的殘差區塊加深（每個階段 4 個 Block）來萃取更深層特徵，同時在所有卷積層加入 L2 正規化 ($5e-4$)，並配合 Global Average Pooling 來限制權重過度生長。
- II. 優化器與學習率：捨棄前期收斂雖快但易過擬合的 Adam，改用泛化能力更好的 SGD (Momentum=0.9)。搭配 Cosine Decay 學習率排程，讓模型在前期大步探索，後期平穩降落。為了讓學習率完整衰減，將 Epochs 設為 250 圈。

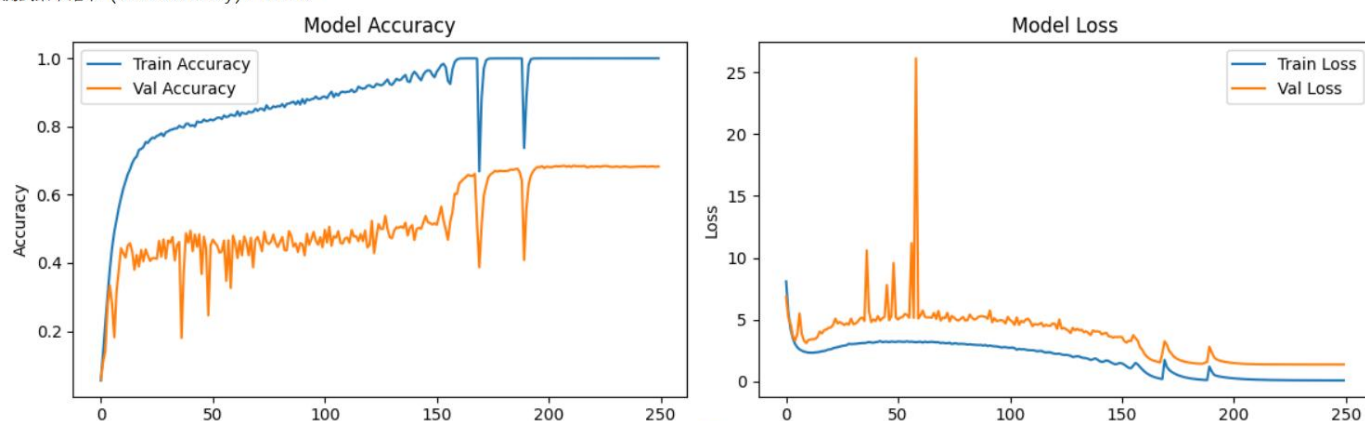
2. 遇到困難及解決方法

- I. 嚴重 Overfitting：初期測試 Adam 優化器時，Training accuracy 很快飆到 95%，但 Validation accuracy 卻死卡在 45% 左右。在無法擴增資料的限制下，我果斷改用 SGD + Cosine Decay，並加大 L2 懲罰，成功在後期將驗證準確率拉升突破 50%。
- II. 硬體算力不足與死線危機：原本打算使用本機 RTX 4050 訓練，但在解決完 CUDA/cuDNN 的環境衝突後，發現跑深層網路一圈要近 3 分鐘，完整訓練需耗時 10 小時。為了解決時間壓力，我立刻改用 Google Colab 並購買運算單元啟用 A100 GPU，將單圈時間壓縮至 10 秒，順利在死線前 15 分鐘跑完 250 圈收斂並產出預測結果。

準確率截圖與更改前後比較之撰寫部分

1. 準確率截圖

測試集準確率 (Test Accuracy): 0.6837



2. 更改前後之比較

I. 引入 L2 正規化 (L2 Regularization)

```
1 # 改動前
2 x = Conv2D(filters=64, kernel_size=(3, 3), padding='same', kernel_regularizer=l2(1e-4), activation='relu')(inputs)
3 # 改動後
4 x = Conv2D(64, (3, 3), padding='same', kernel_regularizer=l2(5e-4))(inputs)
5 x = BatchNormalization()(x)
6 x = Activation('relu')(x)
```

II. 加深殘差網路 (增加 Blocks 數量)

```
1 def residual_block(x, filters, stride=1):
2     shortcut = x
3     if stride != 1 or shortcut.shape[-1] != filters:
4         shortcut = Conv2D(filters, (1, 1), strides=stride, padding='same', kernel_regularizer=l2(5e-4))(shortcut)
5         shortcut = BatchNormalization()(shortcut)
6
7         fx = Conv2D(filters, (3, 3), strides=stride, padding='same', kernel_regularizer=l2(5e-4))(x)
8         fx = BatchNormalization()(fx)
9         fx = Activation('relu')(fx)
10
11        fx = Conv2D(filters, (3, 3), padding='same', kernel_regularizer=l2(5e-4))(fx)
12        fx = BatchNormalization()(fx)
13
14        out = Add()([shortcut, fx])
15        return Activation('relu')(out)
16
17 # 深層架構：每個階段 4 個 Block
18 x = residual_block(x, 128)
19 x = residual_block(x, 128)
20 x = residual_block(x, 128)
21 x = residual_block(x, 128)
22
23 x = residual_block(x, 256, stride=2)
24 x = residual_block(x, 256)
25 x = residual_block(x, 256)
26 x = residual_block(x, 256)
27
28 x = residual_block(x, 512, stride=2)
29 x = residual_block(x, 512)
30 x = residual_block(x, 512)
31 x = residual_block(x, 512)
```

III. 捨棄攤平層，改用全局平均池化



```
1 # 改動前
2 x = Flatten()(x)
3 # 改動後
4 x = GlobalAveragePooling2D()(x)
```

IV. 切换優化器



```
1 # 改動前
2 optimizer = Adam(learning_rate=learning_rate)
3 # 改動後
4 optimizer = tf.keras.optimizers.SGD(learning_rate=learning_rate_schedule, momentum=0.9, nesterov=True)
```

V. 引入餘弦退火學習率與增加圈數



```
1 # 改動前
2 epochs = 15
3 # 改動後
4 epochs = 250
5 learning_rate_schedule = tf.keras.optimizers.schedules.CosineDecay(
6     initial_learning_rate=0.1,
7     decay_steps=(50000 * 0.9 // batch_size) * epochs,
8     alpha=0.001
9 )
```